

AgentBloc V2 ? Prompt de Desarrollo

Contexto

Este documento recoge las conclusiones de una investigación profunda sobre frameworks de agentes de IA (CrewAI, LangGraph, AutoGen/AG2, OpenAI Agents SDK, Google ADK, Mastra, Paperclip, Letta) y define cómo debe evolucionar AgentBloc para convertirse en un diseñador y orquestador de equipos de agentes de IA reales.

AgentBloc es un skill de Claude Code dentro de ClaudeClaw (TypeScript + Bun). NO es un framework independiente ? usa Claude Code como runtime y ClaudeClaw como infraestructura (jobs, Telegram, MCP servers).

Visión: De Automatizador a Consultor de IA Proactivo

AgentBloc NO es un simple automatizador de workflows. Es un consultor de IA que entiende tu negocio, diseña el equipo de agentes que necesitas, y además te sugiere lo que no sabías que necesitabas.

3 Capas de Inteligencia

1. Understand ? Entrevista profunda al usuario para entender su negocio, procesos, dolor, herramientas actuales, canales de comunicación y patrones de decisión. Genera un Business Graph estructurado.
2. Diagnose ? Clasifica cada proceso del negocio en tipos de automatización:
 - Tareas repetitivas ? agente programado (cron)
 - Reacciones a eventos ? agente reactivo (webhook/trigger)
 - Decisiones con datos ? agente analítico
 - Comunicación ? agente de interfaz (Telegram/email)
 - Monitoreo ? agente watchdog (loop)
3. Anticipate ? Sugiere agentes y automatizaciones que el usuario NO pidió pero que necesita. Ejemplo: si gestiona alquileres, sugerir un agente de análisis de rentabilidad aunque solo pidió cobros.

Investigación Competitiva: Qué Robar de Cada Framework

Los 5 Patrones de Orquestación Universales

Todos los frameworks usan variaciones de estos 5 patrones:

Patrón	Cómo funciona	Mejor para
Negociación conversacional	Agentes "hablan" en grupo, un selector decide quién actúa	Tareas que requieren deliberación
Delegación por roles	Manager asigna tareas a workers especializados	Equipos jerárquicos claros
Cadena de handoffs	Agente A transfiere control a B cuando detecta su dominio	Flujos lineales con especialización
Bus de eventos	Agentes suscritos a eventos, se despiertan cuando ocurren	Sistemas reactivos, monitoreo

Qué Robamos de Cada Framework

Framework	Qué robamos	Por qué	Cómo lo implementamos
AG2 CaptainAgent	Generación dinámica de equipos desde descripción de tarea	Es exactamente lo que hace AgentBloc ? un meta-agente q	
Google ADK	Primitivas SequentialAgent / ParallelAgent / LoopAgent	Simples, combinables, cubren el 95% de los patrones de orquest	
LangGraph	Checkpointing automático con estado tipado	Permite pause/resume en workflows largos y recuperación ante fallos	Git com
Mastra	Validación con Zod schemas entre agentes	Contratos tipados garantizan que el output de un agente sirve como input del sigui	
Paperclip	Control plane UX: cola de aprobación + feed en tiempo real + cost tracker	La parte que hacen bien es la gobernanza visua	

Patrones específicos de Paperclip a implementar

Paperclip es open-source (MIT), 30K GitHub stars en 3 semanas. Su fortaleza no es la arquitectura técnica (usa filesystem para comunicación entre agentes ? lento) sino el diseño del control plane:

Patrón	Descripción	Implementación en AgentBloc
Approval queue separada	Decisiones que requieren humano aparecen en superficie propia, no como ruido	Campo <code>requires_human: true</code>
Cost tracking por agente	Tokens + coste en ? por agente/tarea en tiempo real	Añadir <code>token_count</code> y <code>cost_usd</code> al formato de log
Task locking	Un agente por recurso, evita trabajo duplicado	Campo <code>locked_by</code> en <code>state.json</code> de cada agente
Status badges	"Live now" vs dormant ? salud del sistema de un vistazo	<code>last-run.json</code> incluye <code>status: active idle error</code>

Cómo se comunican sus agentes: CEO escribe asignaciones en filesystem ? workers leen y ejecutan ? escriben resultados ? siguiente agente recoge. Más simple pero más lento que nuestro SendMessage. Nosotros somos superiores técnicamente aquí.

Lo que NO Robamos (y por qué)

- Python runtime (CrewAI, LangGraph, AutoGen) ? nuestro stack es TypeScript + Bun
- Servidor de orquestación separado (Paperclip) ? Claude Code ES el runtime, no necesitamos Node.js + Postgres extra
- Filesystem como bus de comunicación (Paperclip) ? ClaudeClaw SendMessage es más rápido y limpio
- Solo topología jerárquica (Paperclip) ? nosotros soportamos también mesh (agentes peer-to-peer)
- Sin integración MCP (Paperclip) ? nosotros: MCP-first en todo
- LLM router (AutoGen SelectorGroupChat) ? demasiada latencia, mejor hardcodear flujos
- Definición manual de agentes (todos) ? AgentBloc los genera automáticamente desde la entrevista

Arquitectura Técnica

Fase 1: Entrevista ? Business Graph

La entrevista (ya existe en SKILL.md) genera un JSON estructurado:

```
{
  "business": {
    "type": "gestión inmobiliaria",
    "size": "7 propiedades",
    "owner": "Pablo"
  },
  "processes": [
    {
      "name": "cobro_alquiler",
      "steps": [
        "verificar pago en BBVA",
        "enviar recordatorio si no ha pagado",
        "generar recibo cuando paga",
        "actualizar registro"
      ],
      "trigger": { "type": "cron", "schedule": "1er día del mes" },
      "tools": ["bbva", "gmail", "google-drive"],
      "pain": "manual, se olvidan pagos, no hay seguimiento",
      "frequency": "mensual",
      "current_actor": "Pablo (manual)"
    },
    {
      "name": "respuesta_consultas",
      "steps": [
        "recibir consulta de inquilino",
        "buscar info del contrato",
        "responder con datos relevantes"
      ],
    }
  ]
}
```

```

    "trigger": { "type": "event", "source": "telegram/email" },
    "tools": ["telegram", "google-drive"],
    "pain": "interrumpe el trabajo, repetitivo",
    "frequency": "diaria",
    "current_actor": "Pablo (manual)"
  }
],
"tools_available": ["gmail", "bbva", "google-drive", "telegram", "n8n"],
"channels": ["telegram", "email"],
"decision_patterns": [
  "si impago > 7 días ? aviso formal",
  "si consulta sobre contrato ? buscar en Drive"
]
}
}

```

Fase 2: Designer Agent ? Perfiles de Agentes

Un agente "Designer" (el corazón de AgentBloc) analiza el Business Graph y genera automáticamente los perfiles de cada agente. Usa el patrón de AG2 CaptainAgent.

Output: Agent Profiles (YAML)

```

team:
  name: "arco-rooms-team"
  topology: "mesh" # pipeline | mesh | hierarchy | swarm

agents:
- id: gestor-cobros
  role: "Gestor de Cobros"
  goal: "Cobrar alquileres puntualmente y gestionar morosidad con mínima intervención humana"
  backstory: "Especialista en gestión de cobros inmobiliarios. Tiene acceso a banca online, email y documentos. Sabe c
  tools:
    - bbva-mcp # consultar movimientos bancarios
    - gmail-mcp # enviar recordatorios y recibos
    - drive-mcp # acceder a contratos y plantillas
  triggers:
    - type: cron
      schedule: "0 9 1 * *" # día 1 de cada mes a las 9am
    - type: event
      source: "bbva"
      event: "payment_received"
    - type: event
      source: "timer"
      event: "overdue_7d" # 7 días sin pago
  autonomy: semi # ask before sending emails to tenants
  outputs:
    - type: "payment_status"
      schema: { tenant: string, amount: number, status: "paid|pending|overdue", date: string }
  escalation: "telegram:pablo" # escalar a Pablo si no puede resolver

- id: recepcionista
  role: "Recepcionista Virtual"
  goal: "Responder consultas de inquilinos con información precisa y actualizada"
  backstory: "Primer punto de contacto para inquilinos. Accede a contratos, historiales de pago y normativas. Responde
  tools:
    - telegram-mcp # recibir y responder mensajes
    - drive-mcp # buscar contratos e info
    - gmail-mcp # responder por email si necesario
  triggers:
    - type: event
      source: "telegram"
      event: "tenant_message"
    - type: event
      source: "gmail"
      event: "tenant_email"
  autonomy: semi
  dependencies:
    - gestor-cobros # puede consultar estado de pagos

```

```

escalation: "telegram:pablo"

- id: analista          # AGENTE ANTICIPADO - el usuario no lo pidió
  role: "Analista de Rentabilidad"
  goal: "Monitorizar la salud financiera de las propiedades y detectar oportunidades o problemas"
  backstory: "Analista financiero inmobiliario. Genera informes periódicos de ocupación, rentabilidad, morosidad y gas"
  tools:
    - drive-mcp
    - bbva-mcp
  triggers:
    - type: cron
      schedule: "0 8 * * 1"    # lunes a las 8am
  autonomy: full
  outputs:
    - type: "weekly_report"
      schema: { period: string, income: number, expenses: number, occupancy: number, alerts: string[] }

orchestration:
  workflows:
    - name: "cobro_mensual"
      type: sequential
      agents: [gestor-cobros]
      steps:
        - action: "verificar_pagos"
        - action: "enviar_recordatorios"
          condition: "hay_impagos"
        - action: "generar_recibos"
          condition: "pagos_recibidos"

    - name: "atencion_inquilinos"
      type: event-driven
      agents: [repcionista, gestor-cobros]
      trigger: "tenant_message"
      flow: |
        recepcionista recibe consulta
        ? si necesita datos de pago ? pide a gestor-cobros
        ? responde al inquilino

    - name: "monitoreo_semanal"
      type: loop
      agents: [analista]
      schedule: "weekly"
      output: "weekly_report ? telegram:pablo"

```

Fase 3: Integration Discovery

Para cada herramienta que necesita un agente, el sistema busca en este orden:

1. Ya existe MCP en `.mcp.json` ? usar directamente
2. Existe MCP en el ecosistema ? instalar (`npx -y @mcp/xxx`)
3. Existe API pública ? generar wrapper MCP con skill `mcp-builder`
4. No existe nada ? usar browser automation (Playwright MCP)

Cada integración se verifica antes de deploy: ¿el MCP responde? ¿tiene los permisos necesarios? ¿devuelve los datos esperados?

Fase 4: Deployment Pipeline

Cada agente se despliega como:

1. Skill de ClaudeClaw ? archivo `skills/{agent-id}/SKILL.md` con el prompt completo del agente, sus herramientas, y sus reglas de autonomía
2. Job de ClaudeClaw ? configuración cron o trigger en el sistema de jobs
3. Config MCP ? entradas necesarias en `.mcp.json`
4. Memoria ? directorio `.claude/agents/{agent-id}/` con:

- `memory.md` ? conocimiento persistente del agente
- `state.json` ? estado actual (datos de trabajo)
- `last-run.json` ? log de última ejecución

Fase 5: Runtime y Gestión Multi-Agente

Scheduling ? Cuándo se despiertan los agentes:

Trigger	Mecanismo	Ejemplo
-----	-----	-----
Evento externo	n8n webhook ? trigger ClaudeClaw	"Pago recibido en BBVA"
Mensaje	Telegram bot ? enruta al agente	"Inquilino pregunta algo"
Inter-agente	SendMessage entre agentes	"Recepcionista pide datos a gestor-cobros"

| Cron

Webhooks y eventos en tiempo real (vía n8n):

Los agentes NO dependen solo de cron. n8n actúa como event bus para triggers en tiempo real:

Evento	Fuente	Webhook n8n	Agente que se activa
-----	-----	-----	-----
Pago entrante detectado	BBVA/Plaid	Plaid webhook ? n8n	Gestor de Cobros
Nuevo formulario de contacto	Web	Form submit ? n8n webhook	Recepcionista
Factura recibida (Digi, Aqualia...)	Gmail	Gmail filter ? n8n	Gestor Documental
Incidencia reportada	Telegram/email	Message trigger ? n8n	Gestor de Incidencias
Cambio en calendario	Google Calendar	Calendar watch ? n8n	Agente de Agenda

| Email

Flujo: evento ocurre ? n8n recibe webhook ? n8n ejecuta ClaudeClaw job del agente correspondiente ? agente actúa inmediatamente.

Esto garantiza respuesta en tiempo real sin esperar al próximo ciclo de cron. El cron es para tareas programadas (informes diarios, verificaciones periódicas). Los webhooks son para reacciones inmediatas.

Coordinación ? Cómo trabajan juntos:

- Tareas de un solo agente: el agente ejecuta solo, sin coordinación
- Tareas multi-agente: `TeamCreate` crea equipo temporal ? agentes se coordinan por `SendMessage` ? equipo se disuelve al terminar
- El primer agente que detecta que necesita a otro spawna el equipo

Estado ? Cómo recuerdan:

```
.claude/agents/  
gestor-cobros/  
  memory.md      # conocimiento persistente (inquilinos, contratos)  
  state.json      # estado actual (pagos del mes)  
  last-run.json   # cuándo corrió, qué hizo, resultado  
recepcionista/  
  memory.md  
  state.json  
  last-run.json  
analista/  
  memory.md  
  state.json  
  last-run.json
```

Cada agente al despertar lee su memoria y estado. Al terminar, los actualiza.

Monitoreo y Reporting (diseñado para escalar a 30+ agentes):

Cada agente escribe logs estructurados (JSON estandarizado). La capa de visualización es pluggable ? los agentes no saben ni les importa cómo se muestran sus logs.

Formato de log estándar:

```
{  
  "agent_id": "gestor-cobros",  
  "team": "arco-rooms",
```

```

"action": "payment_verified",
"result": "success",
"details": "Piso 4, 850?, María López",
"timestamp": "2026-04-20T09:01:00Z",
"requires_human": false,
"priority": "info",
"token_count": 1240,
"cost_usd": 0.004,
"locked_by": null
}

```

Agent Registry (archivo central):

```

# .claude/agents/registry.yaml
teams:
  arco-rooms:
    lead: gestor-cobros
    agents: [gestor-cobros, recepcionista, analista, gestor-docs, gestor-incidencias]
    reporting_hierarchy:
      gestor-cobros: [gestor-docs]          # gestor-docs reporta a gestor-cobros
      recepcionista: [gestor-incidencias]    # incidencias reporta a recepcionista
      analista: []                          # reporta directamente al dashboard
    dashboard_agent: briefing-agent         # el único que habla al humano

```

Reporting jerárquico (para 15-30+ agentes):

```

30 agentes individuales
? reportan a 5 team leads (por dominio)
  ? team leads reportan a 1 dashboard/briefing agent
    ? dashboard agent genera resumen para el humano

```

El humano solo ve:

- Briefing diario consolidado
- Alertas que requieren decisión
- Dashboard on-demand cuando quiere profundizar

Los agentes individuales NUNCA hablan directamente al humano (excepto escalaciones críticas).

Capas de presentación progresivas:

```

| Fase | Interfaz | Para cuántos agentes | Qué muestra |
|-----|-----|-----|-----|

```

```

| 2 ? Producción | Web dashboard | 5-15 agentes | Cards por agente, timeline, drill-down, alertas |
| 3 ? Empresarial | Management UI | 15-30+ agentes | CRUD agentes, org chart visual, métricas, multi-tenant |

```

Todas las fases leen del mismo formato de log + registry. Cambiar de Telegram a dashboard no requiere tocar los agentes ? solo la capa de visualización.

Event Bus centralizado:

Todos los logs van a un directorio centralizado:

```

.claude/agents/
logs/
  2026-04-20/
    gestor-cobros.jsonl  # append-only, un JSON por línea
    recepcionista.jsonl
    analista.jsonl
  registry.yaml         # agentes activos, equipos, jerarquía

```

Fase 1: archivos JSONL (simple, versionable con git).

Fase 2: base de datos SQLite o similar (queries, dashboards).

Fase 3: sistema de eventos real (n8n, Redis streams, etc.).

Escalación y Autonomía:

Cada agente tiene un nivel de autonomía:

- `full` ? actúa sin preguntar (ej: generar informes internos)
- `semi` ? actúa pero confirma antes de acciones externas (ej: enviar email a inquilino)

- `supervised` ? propone acción y espera aprobación (ej: iniciar proceso legal)

Si un agente no puede resolver algo ? escala a Pablo por Telegram con contexto completo (qué intentó, por qué falló, qué opciones hay).

Qué ya existe y se reutiliza

Componente	Estado	Dónde
Agent spawning	Nativo	Claude Code `Agent` tool
Team coordination	Nativo	Claude Code `TeamCreate` + `SendMessage`
Job scheduling	Existe	en ClaudeClaw Jobs system (cron)
MCP servers	Configurados	`.mcp.json` (Gmail, Drive, BBVA, n8n, etc.)
Telegram bot	Existe	ClaudeClaw daemon
Memory system	Existe	`MEMORY.md` pattern

Qué hay que construir nuevo

Componente	Descripción	Prioridad
Designer Agent	Analiza Business Graph, genera perfiles de agentes + orchestration plan	P0
Integration Discovery	Busca/instala/crea MCPs para cada herramienta que necesita un agente	P0
Orchestration Classifier	Clasifica cada workflow como Sequential/Parallel/Loop/Event-driven	P1
Deploy Pipeline	Genera skills + jobs + configs + memory dirs automáticamente	P1
Agent Memory System	Directorio por agente con memory.md + state.json + last-run.json	P1
Autonomy Controller	Gestiona niveles de autonomía (full/semi/supervised) y escalación	P2
Agent Monitor	Genera briefings diarios y dashboards semanales	P2
Anticipation Engine	Sugiere agentes que el usuario no pidió pero necesita	P2

Flujo End-to-End: Del Usuario al Equipo de Agentes

- Usuario ejecuta /agentbloc
- Interview Engine hace preguntas profundas sobre el negocio
- Business Graph Generator estructura las respuestas en JSON
- Designer Agent analiza el Business Graph:
 - Identifica procesos automatizables
 - Agrupar tareas en roles de agente
 - Define cada agente (role + goal + backstory + tools + triggers)
 - Clasifica orquestación (Sequential/Parallel/Loop)
 - ANTICIPA: sugiere agentes extras que el usuario necesita
- Presenta el equipo propuesto al usuario para confirmación
- Usuario aprueba (puede modificar)
- Integration Discovery verifica/instala/crea MCPs necesarios
- Deploy Pipeline genera:
 - skills/{agent-id}/SKILL.md para cada agente
 - Jobs de ClaudeClaw para triggers
 - Configs en .mcp.json
 - Directorios de memoria por agente
- Equipo operativo ? agentes se despiertan según triggers y trabajan
- Monitoreo continuo ? briefings y alertas a Pablo por Telegram

Stack Técnico Definitivo

Claude Code CLI	? runtime de agentes (usa suscripción Max o API key)
ClaudeClaw	? scheduling (cron jobs) + Telegram bot + hooks
n8n	? event bus + webhooks en tiempo real
Archivos	? estado persistente (JSON/YAML/MD/JSONL)

Decisiones de Stack y Justificación

¿Por qué Claude Code y no un servicio custom con Claude API?

- Claude Code CLI puede usar la suscripción Max (\$200/mes) ? sin coste extra por tokens
- Incluye todas las herramientas: MCP, archivos, bash, web, subagentes
- Cuando la escala lo requiera, se cambia `ANTHROPIC\_API\_KEY` en `.env` y pasa a pay-per-token sin tocar nada más

¿Por qué ClaudeClaw y no Paperclip u otro orchestrator?

- ClaudeClaw es un plugin de Claude Code ? todo corre en el mismo proceso
- Ya tiene: job scheduler (cron), Telegram bot, hooks, skills system
- Paperclip requiere servidor Node.js + Postgres 24/7 separado, no usa MCP, solo topología jerárquica
- Los primitivos de orquestación ya existen en Claude Code: `TeamCreate`, `SendMessage`, `Agent` tool

¿Por qué n8n como event bus?

- Ya está desplegado en la infraestructura de Pablo
- Recibe webhooks nativamente (Gmail watch, Plaid, formularios, calendarios)
- Puede disparar jobs de ClaudeClaw cuando ocurre un evento externo
- Cero código nuevo necesario para la capa de eventos

Escala: de suscripción a API

Fase	Agentes	Billing	Coste estimado

| Producción | 5-15 | API key (Sonnet/Haiku) | ~\$50-150/mes (variable) |
| Empresarial | 15-30+ | API key + batch + cache | ~\$200-500/mes (optimizado) |

| Valid

Con prompt caching (90% descuento en tokens repetidos) y batch API (50% descuento), los costes de API se controlan.

Restricciones de Diseño

- No dependencias externas de frameworks ? todo corre sobre Claude Code + ClaudeClaw + n8n
- Switch transparente suscripción?API ? una variable de entorno, cero cambios en código
- UI progresiva ? Fase 1: Telegram. Fase 2: Web dashboard (Bun + Hono). Fase 3: Management UI. Diseñar para escalar desde día 1
- Datos primero, UI después ? logs estructurados JSON + registry YAML como fuente de verdad. La presentación es pluggable
- No base de datos en Fase 1 ? archivos JSONL para logs, JSON/YAML/MD para estado (simple, versionable con git). SQLite en Fase 2
- No servidor 24/7 ? los agentes duermen y se despiertan por triggers
- TypeScript + Bun ? no Python, no Node.js tradicional
- MCP first ? toda integración externa via MCP servers
- Human-in-the-loop ? siempre hay un nivel de escalación a humano

Ejemplo Concreto: Arco Rooms

Para validar AgentBloc v2, usamos Arco Rooms (gestión de 7 pisos de alquiler) como caso de prueba.

Equipo que AgentBloc debería generar automáticamente:

1. Gestor de Cobros ? verifica pagos BBVA, envía recordatorios, genera recibos
2. Recepcionista Virtual ? responde consultas de inquilinos por Telegram/email
3. Gestor Documental ? organiza contratos, facturas, recibos en Drive
4. Analista de Rentabilidad ? informes semanales de ocupación y finanzas (ANTICIPADO)

5. Gestor de Incidencias ? recibe y trackea problemas reportados por inquilinos (ANTICIPADO)

Los agentes 4 y 5 son los que AgentBloc sugiere proactivamente ? el usuario solo pidió cobros y respuestas a consultas, pero el sistema detecta que la gestión inmobiliaria necesita también análisis financiero y tracking de incidencias.